AI Sales Demo Generator

Stateful Architecture & Technical Implementation

Architecture

PostgreSQL + Redis + Workers

Stack

Python FastAPI + Celery

AI

OpenAI GPT-4 Integration

Technical Architecture Document v1.0

Core Architecture - Stateful Design

PostgreSQL Database

users: id, email, api_key, created_at

demos: id, user_id, title, status, created_at

slides: id, demo_id, slide_num, content jsonb

templates: id, name, structure jsonb

generation_jobs: id, demo_id, status, progress

Redis Cache Layer

- User sessions & JWT tokens
- Job queue status tracking
- Template caching (JSONB)
- Rate limiting counters
- Celery message broker

Application Server - FastAPI

REST API: POST /api/generate-demo, GET /api/status/{job_id}

Auth: JWT with Bearer tokens, API key rotation

Async: asyncio for I/O, background tasks

ORM: SQLAlchemy 2.0, connection pooling

Background Workers - Celery

Tasks: scrape_website, parse_linkedin, generate_slides

Queue: Redis as broker + result backend

Concurrency: 4 workers, prefetch_multiplier=2

Retry: 3 attempts with exponential backoff

Monitoring

Prometheus + Grafana, structured logging

Storage

S3 or local disk with CloudFront CDN

Technical Implementation

FastAPI Main Endpoint

```
@app.post("/api/generate-demo")
async def generate_demo(request: DemoRequest):
  job = await create_job(request)
  task = generate_task.delay(job.id)
  return {"job_id": job.id, "status": "queued"}
```

SQLAlchemy Model

```
class Demo(Base):
  __tablename__ = "demos"
  id = Column(UUID, primary_key=True)
  slides = Column(JSONB) # slide metadata
  status = Column(String, default="pending")
```

Redis Integration

```
redis = Redis(host='localhost', db=0)
redis.setex(f"session:{user_id}", 3600, token)
job_status = redis.get(f"job:{job_id}:status")
```

Celery Worker Task

```
@celery.task
def generate_task(job_id):
  demo = fetch_job_data(job_id)
  content = scrape_website(demo.url)
  slides = ai_generate(content)
  html = render_slides(slides)
  save_to_db(job_id, slides)
```

JWT Authentication

```
def create_token(user_id):
  payload = {"user": user_id, "exp": datetime + 3600}
  return jwt.encode(payload, SECRET_KEY)
```

OpenAI Integration

```
client = OpenAI(api_key=os.environ["OPENAI_KEY"])
response = client.chat.completions.create(
  model="gpt-4",
  messages=[{"role": "user", "content": prompt}]
)
```

Data Processing Pipeline

Input Validation & Sanitization

```
def validate_input(data: dict):
  if not validators.url(data['url']):
    raise ValidationError("Invalid URL")
  clean = bleach.clean(data['text'])
  return sanitize_input(clean)
```

Web Scraping - Playwright

```
async def scrape_website(url):
  browser = await playwright.chromium.launch()
  page = await browser.new_page()
  await page.goto(url, timeout=30000)
  content = await page.content()
  soup = BeautifulSoup(content, 'html.parser')
  return extract_text(soup)
```

LinkedIn API Integration

```
def fetch_linkedin(url, token):
  headers = {'Authorization': f"Bearer {token}"}
  resp = requests.get(url, headers=headers)
  profile = resp.json()
  return parse_profile(profile)
```

AI Prompt Engineering

```
PROMPT_TEMPLATE = """
  Generate 10 slide deck content from:
  Website: {website_content}
  LinkedIn: {linkedin_profile}
  Founder: {founder_background}
  Focus: {about_me}
  Output: JSON with title, bullets per slide"""
```

HTML/Tailwind Generation

```
def render_slides(slides_data):
  env = Environment(loader=FileSystemLoader('templates'))
  template = env.get_template('slide.html')
  for slide in slides_data:
    html = template.render(**slide)
    save_slide(html, slide['id'])
```

Error Handling & Retry

```
@retry(stop=stop_after_attempt(3),
   wait=wait_exponential(min=1, max=10))
def ai_generate(content):
  try:
    return openai.chat(prompt, content)
  except OpenAIError as e:
    logger.error(f"AI failed: {e}")
    raise
```

Deployment & Scaling

Docker Configuration

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
CMD ["gunicorn", "-w 4", "-k uvicorn.workers.UvicornWorker"]
```

PostgreSQL Cluster

```
docker-compose.yml:
  postgresql: # Primary + 2 replicas
    environment:
      POSTGRES_MAX_CONNECTIONS: 200
      POSTGRES_SHARED_BUFFERS: 256MB
  pgbouncer: # Connection pooler
    max_client_conn: 1000
    default_pool_size: 50
```

Nginx Configuration

```
upstream fastapi_backend {
  least_conn;
  server app1:8000;
  server app2:8000;
  server app3:8000;
}
server {
  location / { proxy_pass http://fastapi_backend; }
}
```

GitHub Actions CI/CD

```
jobs:
  build:
    # Build & push Docker image
  test:
    run: pytest --cov=app tests/
  deploy:
    run: docker-compose up -d
    run: kubectl apply -f k8s/
```

Monitoring Stack

```
prometheus.yml:
  scrape_configs:
    - job_name: 'fastapi'
      metrics_path: /metrics
    - job_name: 'celery'
    - job_name: 'postgres_exporter'
Grafana Dashboards: System, API, Queue
```

Backup & Scaling

```
#!/bin/bash
pg_dump -Fc app_db > /backups/dump_$(date +%F).dump
aws s3 sync /backups s3://demo-backups/

Scaling:
  Helm upgrade --replicas=5 app ./chart
  kubectl autoscale --max=10
```